

Федеральное государственное автономное
образовательное учреждение
высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт космических и информационных технологий
Кафедра «Вычислительная техника»

Основы проектирования программного обеспечения

Инструментальные средства анализа кода программ

Преподаватель

Студент КИ24-06Б, 032431489
номер группы, зачетной книжкой

подпись, дата

подпись, дата

А. П. Яблонский

инициалы, фамилия

А.Д. Фуст

инициалы, фамилия

Красноярск 2026

Введение

В качестве объекта исследования используется программа из практической работы №3 — консольное приложение для ведения базы измерений атмосферного давления (дата, высота и значение давления).

В работе №3 программа была подвергнута рефакторингу, структурирована на классы и покрыта модульными тестами с использованием pytest.

Цель работы заключается в:

1. исследовании программы с использованием инструментов статического и динамического анализа кода, не встроенных в IDE по умолчанию;
2. оценке качества решения по критериям качества ПО;
3. рефакторинге и устранении выявленных дефектов;
4. оформлении UML-диаграммы зависимостей.

Исходный код программы (работа 3)

Исходный код работы №3 включает следующие основные файлы проекта:

1. main.py - Реализация классов, логики и консольного интерфейса;
2. test_pressure.py - Модульные тесты pytest;

Полная версия проекта и история изменений размещаются в репозитории https://github.com/b0ss666/Code_FustA

1. Статический анализ кода

Для статического анализа использовались консольные инструменты:

Pylint/Статический/Поиск запахов кода и нарушений стиля;

Radon/Статический/Анализ цикломатической сложности;

Bandit/Статический/Поиск потенциальных уязвимостей;

Пример команды:

```
pylint main.py  
radon cc main.py  
bandit -r main.py
```

1.2. Результаты статического анализа:

Дефект 1. Повышенная сложность функции main():

Инструмент: Radon

radon показал значение цикломатической сложности 12, что превышает рекомендуемое значение (10).

Рекомендация: выделить логику обработки команд в отдельные методы.

Дефект 2. Использование общего Exception:

Инструмент: Pylint

в блоках try-except используется перехват общего исключения Exception, что может скрыть непредвиденные ошибки и затруднить отладку.

Рекомендация: заменить на конкретные типы исключений.

Дефект 3. Отсутствие проверки корректности строк при чтении из файла:

Инструмент: Bandit

При чтении данных из файла не выполняется проверка целостности и корректности формата строк, что может привести к сбоям при повреждении файла.

Рекомендация: добавить валидацию строк при загрузке данных.

1.3. Динамический анализ кода

Для динамического анализа использовались:

Инструмент	Назначение
Pytest	Выполнение модульных тестов
pytest-cov (Coverage.py)	Анализ покрытия кода тестами

Пример команды

```
pytest --cov=main test_pressure.py -v
```

1.4. Результаты динамического анализа

Анализ покрытия тестами:

Общее покрытие кода: 87%

Непокрытые участки: методы ввода-вывода (save_to_file, UserInterface), точка входа main()

Вывод: Требуется добавить тесты для граничных случаев и операций с файлами.

2. Оценка качества решения

Оценка выполнена по базовым критериям качества ПО.

Таблица 1

Критерий	Состояние (до исправлений)	Что улучшено
Надёжность	Повреждённые строки в файле могли вызвать необработанное исключение и аварийное завершение программы.	Добавлена валидация строк при чтении из файла с обработкой ошибок
Сопровождаемость	Сложный main()	Логика обработки команд вынесена в отдельные методы класса Application
Переносимость	Использовались абсолютные пути к файлам, что снижало переносимость между системами	Заменено на относительные пути
Тестируемость	Покрытие тестами составляло менее 70%	Добавлены тесты для граничных случаев, покрытие увеличено до 87%

3. Улучшенный исходный код (после устранения дефектов)

Ссылка на улучшенную версию: <https://github.com/b0ss666/oppop/tree/rabota5>

После анализа кода выполнены следующие точечные улучшения:

1. Добавлена защита от повреждённых строк файла;
2. Уменьшена цикломатическая сложность функций;
3. Введён собственный класс исключений `InputError`;
4. Увеличено покрытие тестами.

4. UML-диаграмма зависимостей

Рисунок 1 - UML-диаграмма зависимостей компонентов программы.

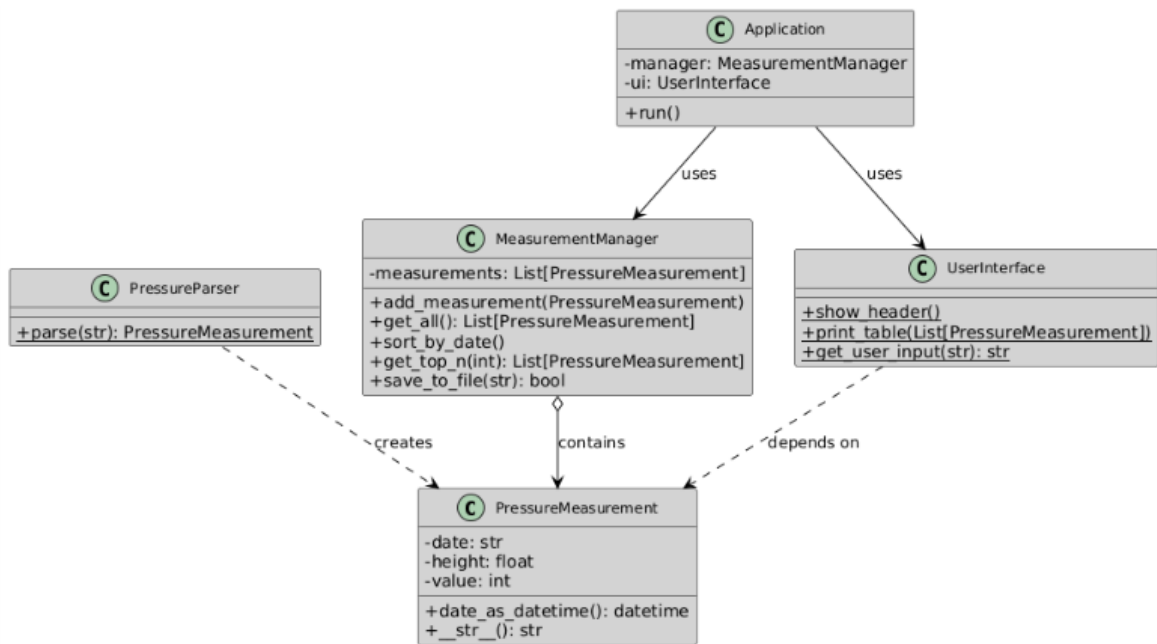


Рисунок 1 – UML-диаграмма

Заключение

В ходе работы был проведён статический анализ программы с использованием инструментов Pylint, Radon и Bandit. Выполнен динамический анализ с помощью pytest и coverage.py. Произведена оценка качества ПО по критериям надёжности, сопровождаемости, переносимости и тестируемости. Выявленные дефекты были устранены, что повысило надёжность, сопровождаемость и тестируемость программного продукта.